LAPORAN PRAKTIKUM PEKAN 1

MATA KULIAH STRUKTUR DATA

DOSEN PENGAMPU :

Dr. WAHYUDI, S.T., M.T.

OLEH :

MUHAMMAD HANS NAFIS

NIM : 2511532027

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur kepada Allah SWT atas segala rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan Laporan Praktikum Pekan 1 ini tepat pada waktunya. Laporan ini disusun untuk memenuhi salah satu tugas praktikum mata kuliah Basis Data, dengan topik pembahasan mengenai konsep dasar *Abstract Data Type* (ADT).

Penyusunan laporan ini bertujuan untuk mendokumentasikan hasil praktikum yang telah dilakukan, sekaligus sebagai saran untuk memperdalam pemahaman penulis terhadap materi yang telah dipelajari selama kegiatan praktikum berlangsung. Dalam proses penyusunan laporan ini, penulis memperoleh banyak pengalaman dan pengetahuan baru, khususnya dalam memahami konsep dasar struktur data serta penerapannya dalam pemograman menggunakan bahasa Java.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan di masa mendatang. Akhir kata, penulis mengucapkan terima kasih kepada Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu mata kuliah Struktur Data, asisten dosen, teman-teman praktikum dan pihak lain yang turut mendukung penulisan laporan ini.

Selasa, 31 Maret 2026

Muhammad Hans Nafis

DAFTAR ISI

KATA PENGANTAR i

DAFTAR ISI ii

BAB 1 1

PENDAHULUAN 1

1.1 Latar Belakang 1

1.2 Tujuan Praktikum 1

1.3 Manfaat Praktikum 1

BAB 2 2

PEMBAHASAN 2

2.1 *Abstract Data Type* (ADT) 2

2.2 Class Jam 2

2.3 Class JamDriver 5

2.4 Class JamDriver2 6

2.5 Tugas Praktikum Pekan 1 9

BAB 3 14

KESIMPULAN 14

DAFTAR PUSTAKA 15

LAMPIRAN 16

BAB 1
PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi yang semakin pesat menuntut adanya pengolahan data yang cepat, tepat, dan efisien. Dalam dunia pemrograman, pengelolaan data yang baik sangat bergantung pada pemahaman terhadap struktur data. Struktur data merupakan cara untuk menyimpan, mengatur, dan mengelola data sehingga dapat digunakan secara optimal oleh suatu program.

Salah satu konsep fundamental dalam struktur data adalah *Abstract Data Type* (ADT). ADT merupakan suatu konsep yang mendefinisikan tipe data berdasarkan perilaku atau operasi-operasi yang dapat dilakukan terhadap data tersebut, tanpa harus mengetahui bagaimana implementasi internalnya. Dengan kata lain, ADT lebih menekankan pada apa yang dapat dilakukan terhadap data, bukan bagaimana data tersebut disimpan atau diimplementasikan.

1.2 Tujuan Praktikum

Tujuan dari pelaksanaan praktikum antara lain sebagai berikut :

1. Memahami konsep dasar *Abstract Data Type* (ADT).
2. Mengetahui cara implementasi ADT dalam bahasa pemrograman Java.

1.3 Manfaat Praktikum

Manfaat dari pelaksanaan praktikum antara lain sebagai berikut :

1. Menambah pemahaman mengenai konsep dasar struktur data, khususnya ADT.
2. Meningkatkan kemampuan pemograman dalam bahasa Java.
3. Melatih keterampilan dalam mengelola kode program secara terstruktur.

BAB 2

PEMBAHASAN

2.1 *Abstract Data Type* (ADT)

Abstract Data Type (ADT) merupakan suatu model atau konsep yang mendefinisikan tipe data berdasarkan operasi-operasi yang dapat dilakukan terhadap data tersebut, tanpa menjelaskan bagaimana data tersebut diimplementasikan secara detail. Dengan kata lain, ADT lebih berfokus pada "apa yang dapat dilakukan" terhadap data. Pendekatan ini memungkinkan programmer untuk merancang sistem secara lebih fleksibel, karena implementasi dapat disesuaikan tanpa mengubah konsep dasarnya.

Dalam implementasinya, ADT biasanya dipresentasikan melalui dua komponen utama, yaitu nilai data dan operasi. Nilai data mempresentasikan informasi yang disimpan, sementara operasi merupakan fungsi atau metode yang dapat digunakan untuk memanipulasi data tersebut. Pada pemograman berorientasi objek, seperti bahasa Java, konsep ADT dapat diimplementasikan melalui class dan method. Class digunakan untuk mempresentasikan tipe data, sedangkan method digunakan untuk mendefinisikan operasi yang dapat dilakukan terhadap data tersebut. Dengan pendekatan ini, ADT membantu dalam menciptakan kode program yang lebih terstruktur, mudah dipahami, serta mudah untuk dikembangkan dan dipelihara.

2.2 Class Jam

Berikut kode program Java mengenai class Jam :

```
public final class Jam_2511532027 {
	private int hh; // 0..23
	private int mm; // 0..59
	private int ss; // 0..59

	// --- Validator ---
	public static boolean isValid(int h, int m, int s) {
		return (0 <= h && h <= 23) && (0 <= m && m <= 59) && (0 <=
s && s <= 59);
	}

	// --- Konstruktor/Kreator ---
```

```java
        public Jam_2511532027(int h, int m, int s) {
                this.hh = h; this.mm = m; this.ss = s;
        }

        // --- Selektor ---
        public int getHour() { return hh;}
        public int getMinute() { return mm;}
        public int getSecond() { return ss;}

        // --- Mutator ---
        public void setHour(int h) {this.hh = h;}
        public void setMinute(int m) {this.mm = m;}
        public void setSecond(int s) {this.ss = s;}

        // --- Konversi ---
        public int toSeconds() { return hh * 3600 + mm * 60 + ss;}
        public static Jam_2511532027 fromSeconds(int total) {
                if (total < 0) throw new IllegalArgumentException("detik
negatif");
                total %= 24 * 3600; // wrap 24 jam
                int h = total / 3600; total %= 3600;
                int m = total / 60; int s = total % 60;
                return new Jam_2511532027(h, m, s);
        }

        // --- Relasional ---
        public int compareTo(Jam_2511532027 other) { return
Integer.compare(this.toSeconds(), other.toSeconds());}
        public boolean equals(Object o) {
                if (!(o instanceof Jam_2511532027 j)) return false;
                return hh == j.hh && mm == j.mm && ss == j.ss;
        }
        public int hashCode() { return java.util.Objects.hash(hh, mm,
ss);}

        // --- Aritmatika ---
        public Jam_2511532027 plus(Jam_2511532027 other) { return
fromSeconds(this.toSeconds() + other.toSeconds());}
```

```
        public Jam_2511532027 minus(Jam_2511532027 other) { return
fromSeconds(this.toSeconds() + other.toSeconds());}
        public Jam_2511532027 nextSecond() { return
fromSeconds(this.toSeconds() + 1);}
        public Jam_2511532027 nextNSecond (int n) { return
fromSeconds(this.toSeconds() + Math.max(0, n));}
        public Jam_2511532027 prevSecond() { return
fromSeconds(Math.floorMod(this.toSeconds() - 1, 24*3600));}
        public Jam_2511532027 prevNSecond(int n) { return
fromSeconds(Math.floorMod(this.toSeconds() - Math.max(0, n), 24*3600));}

        // --- Durasi (detik) -- bisa negatif jika this > other? sesuai
spesifikasi durasi(JAw, JAkh)
        public static int durasiDetik(Jam_2511532027 jaw, Jam_2511532027
jakh) { return jakh.toSeconds();}
        // overriding ke string
        public String toString() { return String.format("%02d:%02d:%02d",
hh, mm, ss);}
}
```

Kode program Java tersebut merupakan implementasi sebuah kelas bernama Jam_2511532027 yang digunakan untuk merepresentasikan waktu dalam satu hari dengan format jam, menit, dan detik. Kelas ini memiliki tiga atribut utama yaitu hh (jam), mm (menit) dan ss (detik), yang masing-masing memiliki batasan nilai, yaitu jam dari 0 sampai 23, serta menit dan detik dari 0 sampai 59. Untuk memastikan nilai yang dimasukkan sesuai dengan batasan tersebut, disediakan metode statis isValid yang berfungsi sebagai validator.

Kelas ini memiliki konstruktor untuk membuat objek waktu dengan menginisialisasi nilai jam, menit, dan detik. Selain itu, terdapat metode selektor (getHour, getMinute, getSecond) untuk mengambil nilai dari masing-masing atribut, serta metode mutator (setHour, setMinute, setSecond) untuk mengubah nilai atribut tersebut.

Daalam hal konversi, kelas ini menyediakan metode toSeconds untuk mengubah waktu menjadi total detik, serta metode statis fromSeconds untuk mengubah total detik kembali menjadi objek waktu. Metode fromSeconds

menangani kondisi jika jumlah detik melebihi satu hari dengan menggunakan operasi modulo 24 jam.

Untuk operasi perbandingan, terdapat metode compareTo yang membandingkan dua objek berdasarkan jumlah detiknya, serta metode equals dan hashCode yang digunakan untuk membandingkan kesamaan objek dan mendukung penggunaan dalam struktur data berbasis hash.

Kelas ini juga mendukung operasi aritmatika waktu seperti penjumlahan (plus), pengurangan (minus), penambahan satu detik atau beberapa detik (nextSecond, nextNSecond), serta pengurangan satu detik atau beberapa detik (prevSecond, prevNSecond). Metode-metode ini memanfaatkan konversi ke detik untuk mempermudah perhitungan, kemudian hasilnya dikonversi kembali ke format jam.

Kelas ini juga menyediakan metode durasiDetik untuk menghitung selisih waktu dalam satuan detik, meskipun implementasinya hanya mengembalikan nilai waktu akhir dalam detik. metode toString di*override* untuk menampilkan waktu dalam format string yang rapi, yaitu HH:MM:SS dengan dua digit untuk setiap komponen.

2.3 Class JamDriver

Berikut kode program Java mengenai class JamDriver :

```
public class JamDriver_2511532027 {
	public static void main(String[] args) {
		Jam_2511532027 a = new Jam_2511532027(23, 59, 50);
		Jam_2511532027 b = new Jam_2511532027(0, 0, 15);
		System.out.println("a = " + a);
		System.out.println("b = " + b);
		System.out.println("a+b = " + a.plus(b));
		System.out.println("next 20s = " + a.nextNSecond(20));
		System.out.println("durasi(a,b) = " +
Jam_2511532027.durasiDetik(a, b));
		System.out.println("a.compareTo(b) = " + a.compareTo(b));
	}
}
```

Kode program Java tersebut merupakan kelas driver bernama JamDriver_2511532027 yang berfungsi untuk menguji berbagai method yang

terdapat pada kelas Jam_2511532027. Pada method main, dibuat dua objek waktu, yaitu objek a dengan nilai 23:59:50 dan objek b dengan nilai 00:00:15. Kedua objek ini digunakan untuk mendemonstrasikan operasi-operasi yang telah diimplementasikan pada kelas sebelumnya.

Program mencetak nilai dari objek a dan b menggunakan method toString, sehingga waktu ditampilkan dalam format HH:MM:SS. Setelah itu, dilakukan operasi penjumlahan waktu menggunakan method plus, dimana objek a dijumlahkan dengan b, dan hasilnya ditampilkan. Program juga menguji method nextNSecond dengan menambahkan 20 detik ke waktu a, yang menunjukkan bagaimana waktu dapat bertambah dan mengalami perputaran ke hari berikutnya jika melewati batas 24 jam.

Selain itu, method durasiDetik digunakan untuk menghitung durasi waktu antara a dan b dalam satuan detik, meskipun implementasinya belum sepenuhnya sesuai dengan konsep durasi yang sebenarnya. Method compareTo digunakan untuk membandingkan a dan b, dengan hasil berupa nilai negatif, nol, atau positif yang menunjukkan hubungan urutan waktu tersebut. Kelas ini berfungsi untuk memverifikasi bahwa seluruh method pada kelas Jam_2511532027 berjalan sesuai dengan yang diharapkan.

2.4 Class JamDriver2

Berikut kode program Java mengenai class JamDriver2 :

```
import java.util.Scanner;

public class JamDriver2_2511532027 {
	public static void main(String[] args) {
		Scanner input = new Scanner(System.in);
		System.out.println("=== Program Driver Objek Jam ===");
		// 1. Input Jam Pertama
		System.out.println("\n--- Input Jam 1 ---");
		Jam_2511532027 j1 = buatJamDariInput(input);

		// 2. Input Jam Kedua
		System.out.println("\n--- Input Jam 2 ---");
		Jam_2511532027 j2 = buatJamDariInput(input);

		// 3. Menampilkan Data
```

```
            System.out.println("\n--- Hasil Operasi ---");
            System.out.println("Jam 1 (String) : " + j1.toString());
            System.out.println("Jam 2 (String) : " + j2.toString());
            System.out.println("Jam 1 dalam detik : " +
j1.toSeconds());
            System.out.println("Jam 2 dalam detik : " +
j2.toSeconds());

            // 4. Operasi Relasional (Perbandingan)
            int perbandingan = j1.compareTo(j2);
            if (perbandingan > 0) {
                  System.out.println("Status : Jam 1 lebih lambat
(setelah) Jam 2");
            }else if (perbandingan < 0) {
                  System.out.println("Status : Jam 1 lebih awal
(sebelum) Jam 2");
            }else {
                  System.out.println("Status : Jam 1 dan Jam 2
persis");
            }

            // 5. Operasi Aritmatika
            System.out.println("Durasi (J1 ke J2) : " +
Jam_2511532027.durasiDetik(j1,j2) + " detik");
            Jam_2511532027 jNext = j1.nextSecond();
            System.out.println("Jam 1 Detik Berikutnya : " + jNext);
            Jam_2511532027 jPrev = j1.prevSecond();
            System.out.println("Jam 1 Detik Sebelumnya : " + jPrev);

            // 6. Operasi Penjumlahan Jam
            Jam_2511532027 jHasilPlus = j1.plus(j2);
            System.out.println("Hasil J1 + J2 : " + jHasilPlus);
            input.close();
            System.out.println("\nProgram Selesai.");
      }

      /**
```

```java
        * Prosedur pembantu untuk melakukan input dan validasi secara
berulang
        * sampai user memasukkan angka yang benar (0..23, 0..59).
        */
       private static Jam_2511532027 buatJamDariInput(Scanner sc) {
              int h, m, s;
              while (true) {
                     System.out.println("Masukkan Jam (0-23) : ");
                     h = sc.nextInt();
                     System.out.println("Masukkan Menit (0-59) : ");
                     m = sc.nextInt();
                     System.out.println("Masukkan Detik (0-59) : ");
                     s = sc.nextInt();

                     // Memanggil method static isValid dari kelas
Jam_2511532027
                     if (Jam_2511532027.isValid(h, m, s)) {
                            return new Jam_2511532027(h, m, s);
                     }else {
                            System.out.println("[Error] Input tidak
valid! Silahkan ulangi.\n");
                     }
              }
       }
}
```

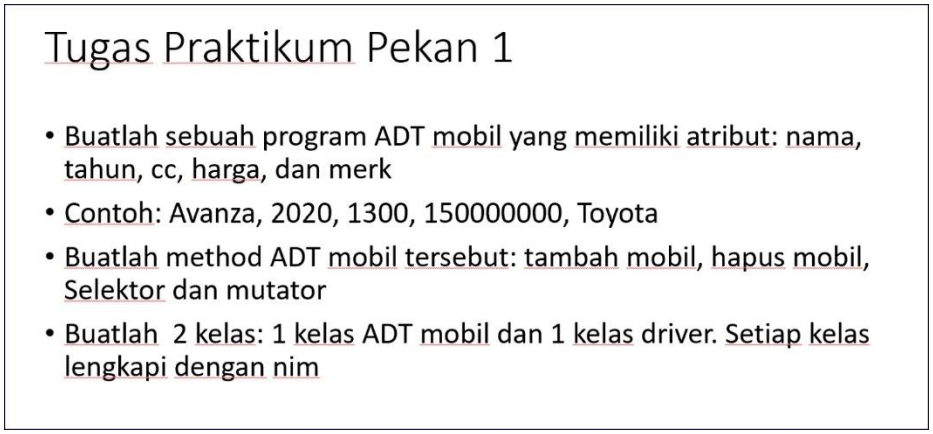
Kode program Java tersebut merupakan kelas driver lanjutan bernama JamDriver2_2511532027 yang berfungsi untuk menguji sekaligus mengimplementasikan penggunaan kelas Jam_2511532027 secara interaktif dengan input dari pengguna. Program dimulai dengan membuat objek Scanner untuk membaca input dari keyboard, kemudian menampilkan judul program. Pengguna diminta untuk memasukkan dua buah waktu (Jam 1 dan Jam 2) melalui pemanggilan method bantu buatJamDariInput, yang akan terus melakukan perulangan hingga pengguna memasukkan nilai jam, menit, detik yang valid sesuai dengan aturan. Validasi ini dilakukan menggunakan method statis isValid dari kelas Jam_2511532027.

Setelah kedua objek waktu berhasil dibuat, program menampilkan informasi dari masing-masing objek, baik dalam bentuk String maupun dalam bentuk total detik menggunakan method toSeconds. Selanjutnya dilakukan operasi relasional dengan method compareTo untuk menentukan apakah Jam 1 lebih awal, lebih lambat, atau sama dengan Jam 2, dan hasilnya ditampilkan dalam bentuk keterangan.

Program juga melakukan beberapa operasi aritmatika waktu, seperti menghitung durasi antara Jam 1 dan Jam 2 menggunakan method durasiDetik, serta menampilkan waktu satu detik setelah (nextSecond) dan satu detik sebelum (prevSecond) dari Jam 1. Selain itu, dilakukan juga operasi penjumlahan waktu antara Jam 1 dan Jam 2 menggunakan method plus. Di akhir, objek Scanner ditutup dan program menampilkan pesan bahwa proses telah selesai. Kelas ini menunjukkan bagaimana penggunaan kelas Jam_2511532027 dalam skenario nyata dengan interaksi pengguna, validasi input, serta penerapan berbagai operasi yang telah diimplementasikan.

2.5 Tugas Praktikum Pekan 1

Berikut tugas praktikum yang diberikan :



Gambar 2.1

Berikut kode program Java dari tugas praktikum :

```
class Mobil_2511532027 {
    private String nama;
    private int tahun;
    private int cc;
    private double harga;
    private String merk;
```

```java
    public Mobil_2511532027(String nama, int tahun, int cc, double
harga, String merk) {
        this.nama = nama;
        this.tahun = tahun;
        this.cc = cc;
        this.harga = harga;
        this.merk = merk;
    }

    public String getNama() {
        return nama;
    }

    public int getTahun() {
        return tahun;
    }

    public int getCc() {
        return cc;
    }

    public double getHarga() {
        return harga;
    }

    public String getMerk() {
        return merk;
    }

    public void setNama(String nama) {
        this.nama = nama;
    }

    public void setTahun(int tahun) {
        this.tahun = tahun;
    }
```

```java
    public void setCc(int cc) {
        this.cc = cc;
    }

    public void setHarga(double harga) {
        this.harga = harga;
    }

    public void setMerk(String merk) {
        this.merk = merk;
    }

    public void tampilkanMobil() {
        System.out.println("Nama  : " + nama);
        System.out.println("Tahun : " + tahun);
        System.out.println("CC    : " + cc);
        System.out.println("Harga : " + harga);
        System.out.println("Merk  : " + merk);
        System.out.println("----------------------");
    }
}
```

Kode program Java tersebut merupakan implementasi sebuah kelas bernama Mobil_2511532027 yang digunakan untuk mempresentasikan data mobil. Kelas ini memiliki lima atribut utama, yaitu nama (nama mobil), tahun (tahun produksi), cc (kapasitas mesin), harga (harga mobil), dan merk (merek mobil). Semua atribut bersifat private, yang berarti tidak dapat diakses langsung dari luar kelas.

Untuk menginisialisasi objek, kelas ini menyediakan konstruktor yang menerima lima parameter sesuai dengan atribut yang dimiliki. Konstruktor ini akan langsung mengisi nilai atribut saat objek dibuat. Selain itu, terdapat method selektor seperti getNama, getTahun, getCc, getHarga, dan getMerk yang berfungsi untuk mengambil nilai dari masing-masing atribut. Method mutator seperti setNama, setTahun, setCc, setHarga, dan setMerk digunakan untuk mengubah nilai atribut setelah objek dibuat.

Kelas ini juga memiliki method tampilkanMobil yang berfungsi untuk menampilkan seluruh informasi mobil ke layar dalam format yang rapi dan

terstruktur. Method ini mencetak setiap atribut beserta nilainya, sehingga memudahkan pengguna dalam melihat mobil secara lengkap.

```
import java.util.ArrayList;

public class DriverMobil_2511532027 {
    static String nim = "2511532027";

    public static void main(String[] args) {

        ArrayList<Mobil_2511532027> daftarMobil = new ArrayList<>();

        Mobil_2511532027 m1 = new Mobil_2511532027("Mercedes Benz C-
Class", 2021, 1991, 779000000, "Mercedes");
        Mobil_2511532027 m2 = new Mobil_2511532027("MX-5 Miata", 2025,
1998, 849000000, "Mazda");
        Mobil_2511532027 m3 = new Mobil_2511532027("86", 2022, 1988,
830000000, "Toyota");
        Mobil_2511532027 m4 = new Mobil_2511532027("WRX", 2026, 2387,
909500000, "Subaru");

        tambahMobil(daftarMobil, m1);
        tambahMobil(daftarMobil, m2);
        tambahMobil(daftarMobil, m3);
        tambahMobil(daftarMobil, m4);

        System.out.println("Daftar Mobil:");
        for (Mobil_2511532027 m : daftarMobil) {
            m.tampilkanMobil();
        }

        hapusMobil(daftarMobil, 0);

        System.out.println("Setelah dihapus:");
        for (Mobil_2511532027 m : daftarMobil) {
            m.tampilkanMobil();
        }
    }
```

```
    public static void tambahMobil(ArrayList<Mobil_2511532027> list,
Mobil_2511532027 mobil) {
        list.add(mobil);
        System.out.println("Mobil berhasil ditambahkan!");
    }

    public static void hapusMobil(ArrayList<Mobil_2511532027> list, int
index) {
        if (index >= 0 && index < list.size()) {
            list.remove(index);
            System.out.println("Mobil berhasil dihapus!");
        } else {
            System.out.println("Index tidak valid!");
        }
    }
}
```

Kode program Java tersebut merupakan kelas driver bernama DriverMobil_2511532027 yang berfungsi untuk menguji penggunaan kelas Mobil_2511532027 dengan memanfaatkan struktur data ArrayList. Di awal, dibuat sebuah ArrayList bernama daftarMobil yang digunakan untuk menyimpan beberapa objek mobil secara dinamis, Selanjutnya, program membuat empat objek mobil (m1, m2, m3, m4) dengan atribut yang berbeda-beda seperti nama, tahun, cc, harga, dan merk.

Objek-objek mobil tersebut ditambahkan ke dalam ArrayList menggunakan method tambahMobil yang berfungsi untuk memasukkan objek mobil ke dalam list sekaligus menampilkan pesan bahwa mobil berhasil ditambahkan. Setelah itu, program menampilkan seluruh data mobil yang ada di dalam daftarMobil dengan menggunakan perulangan for-each, di mana setiap objek mobil akan memanggil method tampilkanMobil untuk mencetak informasi mobil ke layar.

Selanjutnya, program melakukan operasi penghapusan data menggunakan method hapusMobil dengan parameter indeks, dalam hal ini indeks ke-0, yang berarti menghapus elemen pertama dalam list. Method ini juga dilengkapi dengan validasi untuk memastikan indeks yang dimasukkan berada dalam rentang yang benar. Setelah proses penghapusan, program kembali menampilkan isi daftarMobil untuk menunjukkan perubahan data setelah satu objek mobil dihapus.

BAB 3
KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa ADT merupakan konsep dasar yang sangat penting dalam memahami struktur data. ADT memungkinkan pemrogram untuk mendefinisikan tipe data berdasarkan operasi yang dapat dilakukan tanpa harus memahami detail implementasinya. Melalui praktikum ini, mahasiswa tidak hanya memahami konsep ADT secara teoritis, tetapi juga mampu mengimplementasikannya dalam bentuk program menggunakan bahasa Java. Dengan demikian, praktikum ini memberikan dasar yang kuat untuk mempelajari struktur data yang lebih kompleks.

Sebagai saran, diharapkan mahasiswa untuk lebih sering berlatih dalam mengimplementasikan konsep ADT agar pemahaman menjadi lebih mendalam. Selain itu, sebaiknya materi praktikum juga dilengkapi dengan penjelasan teori yang lebih rinci agar mahasiswa dapat memahami konsep sebelum langsung ke tahap implementasi. Mahasiswa juga disarankan untuk aktif bertanya kepada asisten praktikum apabila mengalami kesulitan, serta membiasakan diri dalam mendokumentasikan kode program dengan baik agar lebih mudah dipahami dan dikembangkan di kemudian hari.

DAFTAR PUSTAKA

[1] https://www.geeksforgeeks.org/dsa/abstract-data-types/ [Diakses 31 Maret 2026]

[2] https://id.wikipedia.org/wiki/Abstract_data_type [Diakses 31 Maret 2026]

[3] https://lppm.polytechnic.astra.ac.id/wp-content/uploads/2021/12/MODUL-STRUKTUR-DATA.pdf [Diakses 31 Maret 2026]

LAMPIRAN

Gambar 2.1 9